

"A vertical slice beats five disconnected demos -- API, database, and event, all working together."

MODULE 49

Capstone Backend and Messaging

Implement the CAP-12 vertical slice end to end: validated REST APIs, transaction-safe persistence, a versioned Kafka event, resilient consumption, and automated tests for the Northstar CRM interaction feature.









MODULE 49 OVERVIEW

Implement the CAP-12 vertical slice: validated APIs, transaction-safe persistence, versioned Kafka events, resilient consumption, and automated tests.

Service agents create and update customers; downstream systems must receive audit and customer events reliably. This module builds the Spring Boot and Kafka backend directly against the architecture and ADRs frozen in Module 48.

DURATION & LAB



4 Hours Theory

Layered architecture, API design, validation, exceptions, events, JWT/RBAC review, and testing.



2 Hours Lab

lab49-crm: DTOs, a migration, a transactional service, a REST endpoint, Kafka publish/consume, and tests.



Scenario

Record an interaction for Amina (CUS-1001) with correlation lab-request-001, end to end.

MODULE ARC



Layers First

Controller, service, repository -- each with one job and a clear boundary.



Versioned Events

CustomerInteractionRecordedV1 on crm.customer.interactions.v1, published after commit.



Tests Are Evidence

Unit, MockMvc, JPA, and Kafka IT -- green twice, not lucky once.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No merge without API evidence, persistence proof, versioned event proof, tests, and a documented failure path.

WHY BACKEND ARCHITECTURE MATTERS

The backend must deliver validated APIs, transactions, and Kafka events as a real vertical slice of the CRM -- not a happy-path-only prototype.



Vertical Slice, Not a Layer

CAP-12 crosses API, service, persistence, and messaging -- delivered together or not at all.



Happy-Path-Only Loses Marks

No negatives, no not-found, no validation failure means lost failure-handling marks.



ADR Fidelity

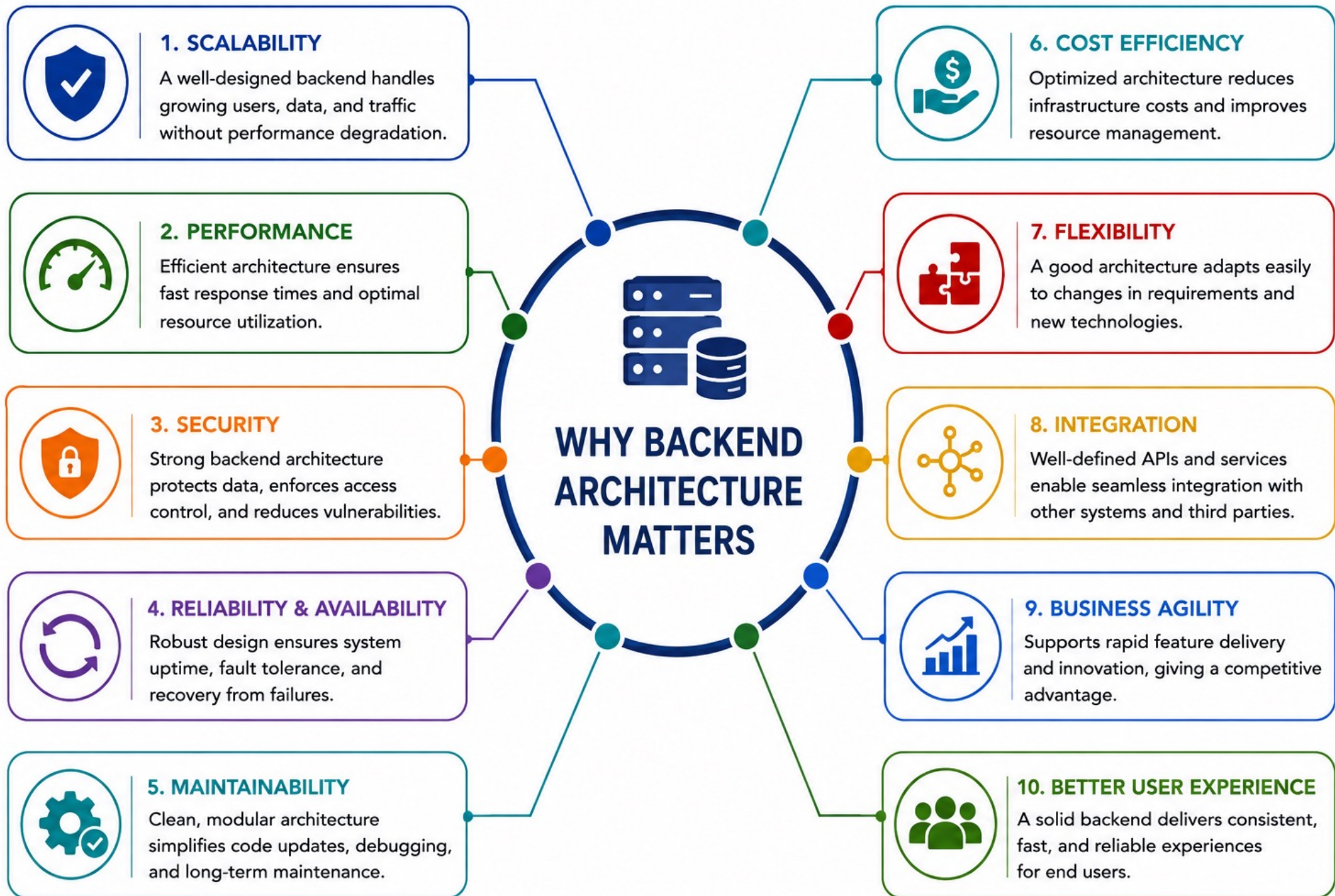
Publish timing and persistence choices must match Module 48's ADRs -- no silent drift.



Graded Like Every Lab

Lab 49 is scored on the same 100-point rubric: core impl, integration, failure handling, verification, security, docs.

KEY TAKEAWAY A green demo without tests, correlation IDs, or failure-path evidence does not pass capstone quality.



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to implement and test a real backend vertical slice for the Northstar CRM.

1. Implement layered Spring Boot features for a CRM vertical slice.
2. Keep JPA entities out of external contracts; validate DTOs.
3. Use transactions deliberately around a persist-then-publish strategy.
4. Publish versioned Kafka events with correlation and actor metadata.
5. Consume events idempotently with bounded retries and a dead letter topic.
6. Test HTTP, persistence, and messaging with failure paths, not just happy paths.
7. Document a reproducible backend demo with synthetic fixtures.
8. Preserve Module 48's ADR decisions faithfully in working code.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will build and test every layer of CAP-12's vertical slice.

SPRING BOOT ARCHITECTURE REVIEW

A quick review of the Spring Boot shape this module builds on -- controllers, services, repositories, and configuration, wired by dependency injection.



Spring Boot App

CrmApplication -- auto-configuration, an embedded server, and component scanning.



Dependency Injection

Constructor injection wires controller
-> service -> repository, testable in isolation.



Configuration

application.yml -- profiles distinguish PostgreSQL from H2 in CI.

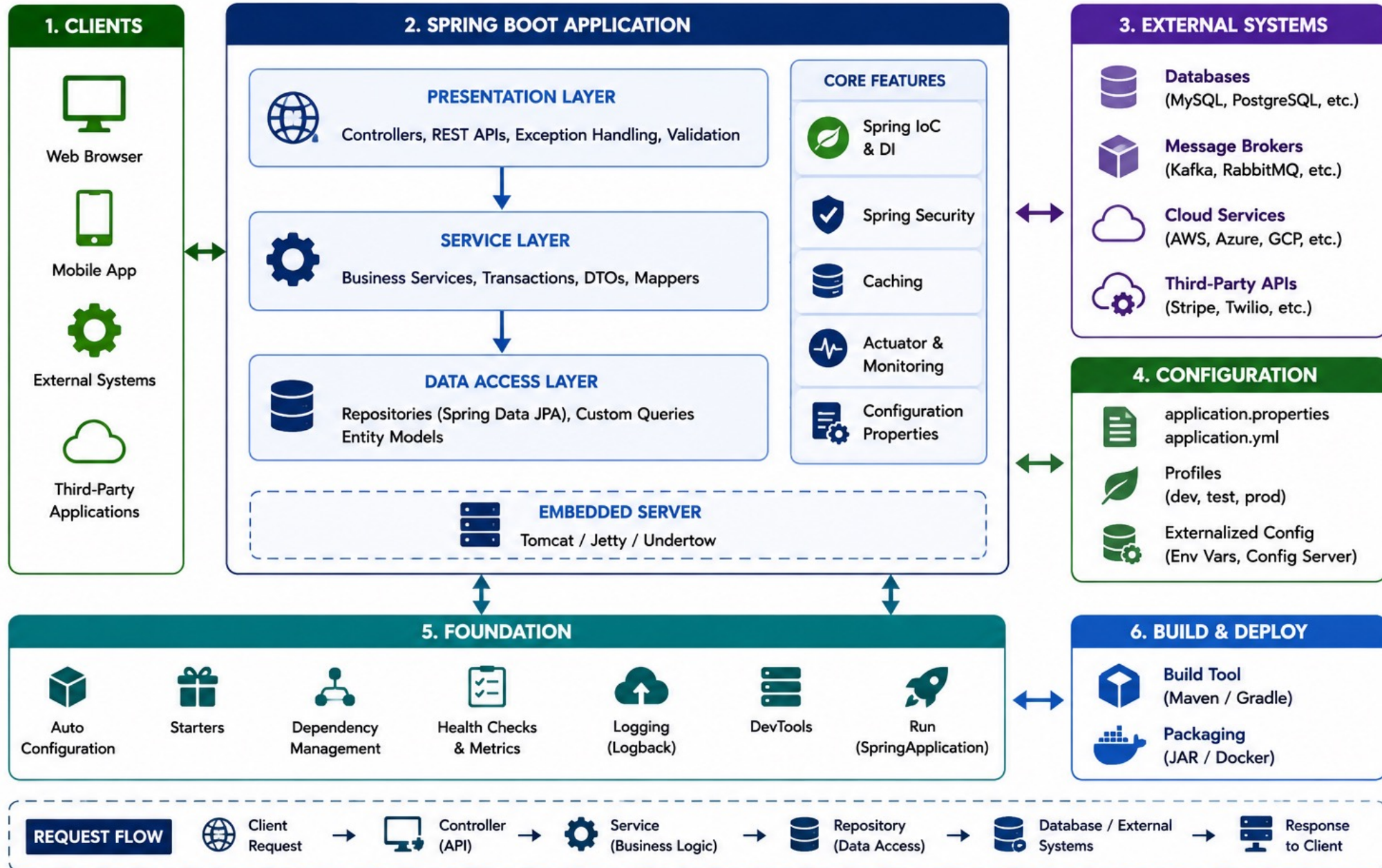


Migrations

Flyway versions the schema -- never 'Hibernate will figure it out'.

KEY TAKEAWAY Everything built this module reuses the same layered Spring Boot shape from earlier weeks -- applied now to a real capstone slice.

SPRING BOOT ARCHITECTURE REVIEW



TRY IT YOURSELF -- EXERCISE 1: SELECT BACKEND VERTICAL SLICE

Reinforces: picking one coherent backend slice for build day, with in/out scope and a fixture plan, before any code is written.

STEPS (notes/lab49-slice-selection.md)

Step	What To Do
1 -- Options	Candidate slices: customer create/read, interaction record, status change + event.
2 -- Check the Reference	The slice must include API + persistence + a Kafka touch if messaging is in scope.
3 -- Fixtures	Define how CUS-1001/CUS-1002 and lab-request-001 appear in demos and tests.
4 -- Out of Scope	Explicitly list what this slice will not include.

Selected slice

```
Slice: record a customer interaction (CAP-12) for CUS-1001.  
In scope: API + persistence + event.   Out of scope: UI (Lab 50).
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-slice-selection.md (workspace: examples/module-49-exercises).

LAYERED ARCHITECTURE REVIEW

Controller, service, repository -- each layer owns one job and hands off cleanly to the next.

LAYER RESPONSIBILITIES

Controller: HTTP mapping, status codes, request/response DTOs.

Service: transactions, business rules, orchestration.

Repository: persistence only -- no HTTP concerns.

Mapper: converts between entities and DTOs at the boundary.

WHAT EACH LAYER AVOIDS

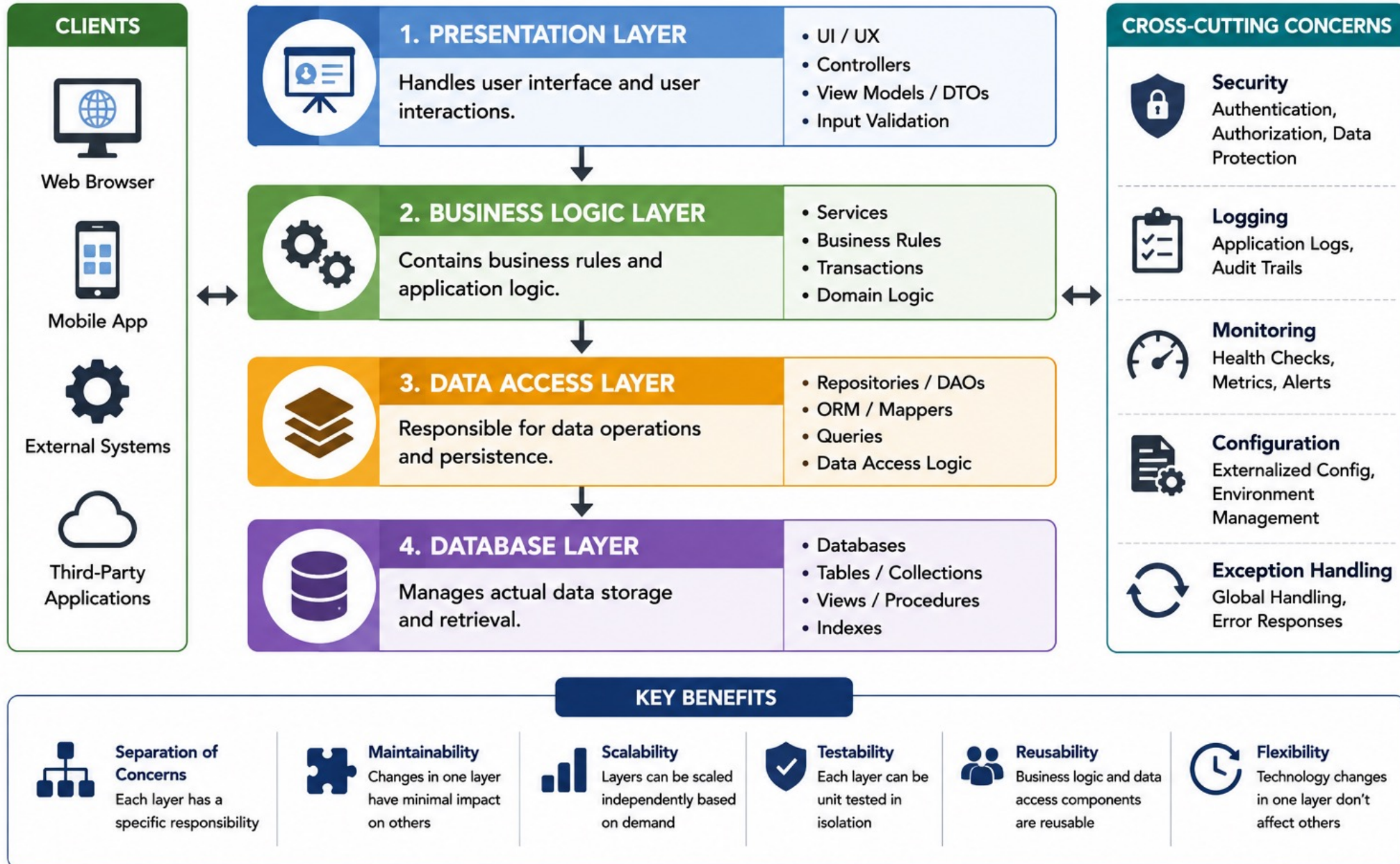
- Controller avoids business-rule sprawl.
- Service avoids raw JDBC or HTTP-specific code.
- Repository avoids validation or transaction orchestration.
- No layer ever returns a JPA entity directly as JSON.

Request flow (CAP-12)

```
POST /api/customers/{id}/interactions  
-> InteractionController -> InteractionService (@Transactional) -> InteractionRepository
```

KEY TAKEAWAY Controllers that open transactions and publish ad hoc skip business rules and correlation -- keep each layer's job separate.

LAYERED ARCHITECTURE REVIEW



TRY IT YOURSELF -- EXERCISE 2: CONTROLLER-SERVICE-REPOSITORY CHECKLIST

Reinforces: listing responsibilities per layer for the chosen slice, and marking which service method needs @Transactional.

STEPS (notes/lab49-layer-checklist.md)

Step	What To Do
1 -- Table	Controller: HTTP/DTO. Service: rules/transactions. Repository: persistence.
2 -- Check the Reference	Validation belongs on inputs; business rules do not live only in controllers.
3 -- Transaction Note	Mark which service method needs @Transactional (placeholder).
4 -- Verify Habit	Note the verify habit: ./mvnw -B test on the backend module.

Layer reference

```
Controller owns: HTTP mapping, status codes.  
Service owns: transactions, domain rules. Repository owns: persistence.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-layer-checklist.md (workspace: examples/module-49-exercises).

SERVICE DESIGN REVIEW

InteractionService is where business rules, transactions, and event publication meet -- and where Module 48's consistency ADR gets honored in code.

SERVICE RESPONSIBILITIES

Load the customer, or throw not-found for CUS-9999.

Map validation and conflict outcomes to typed exceptions.

Persist the interaction inside the transaction.

Publish the event via a collaborator, never a static Kafka client.

TESTING THE SERVICE

- Unit-test with a fake publisher that records the payload.
- Assert the not-found path never calls publish.
- Assert lab-request-001 flows through to the published event.
- No Spring context needed for a pure service unit test.

InteractionService.create (excerpt)

```
@Transactional
public InteractionResponse create(customerId, request, correlationId) {
    var customer = customers.findById(customerId).orElseThrow(...);
    var saved = interactions.save(mapper.toEntity(customer, request));
    events.publish(eventFactory.interactionRecorded(saved, correlationId));
    return mapper.toResponse(saved); }
```

KEY TAKEAWAY Publishing before persisting, with no ADR to justify it, is a stop-and-align moment -- follow Module 48's consistency decision exactly.



REPOSITORY DESIGN REVIEW

Repository methods are named by business need, and constraints live in migrations, not in Hibernate's automatic schema generation.

REPOSITORY DESIGN

findByCustomerIdOrderByCreatedAtDesc for the timeline query.
Avoid N+1 queries when loading a customer's interaction list.
Seed Amina and Ravi via migration or @Sql for deterministic tests.
The foreign key to customer aligns with the Module 48 ADR.

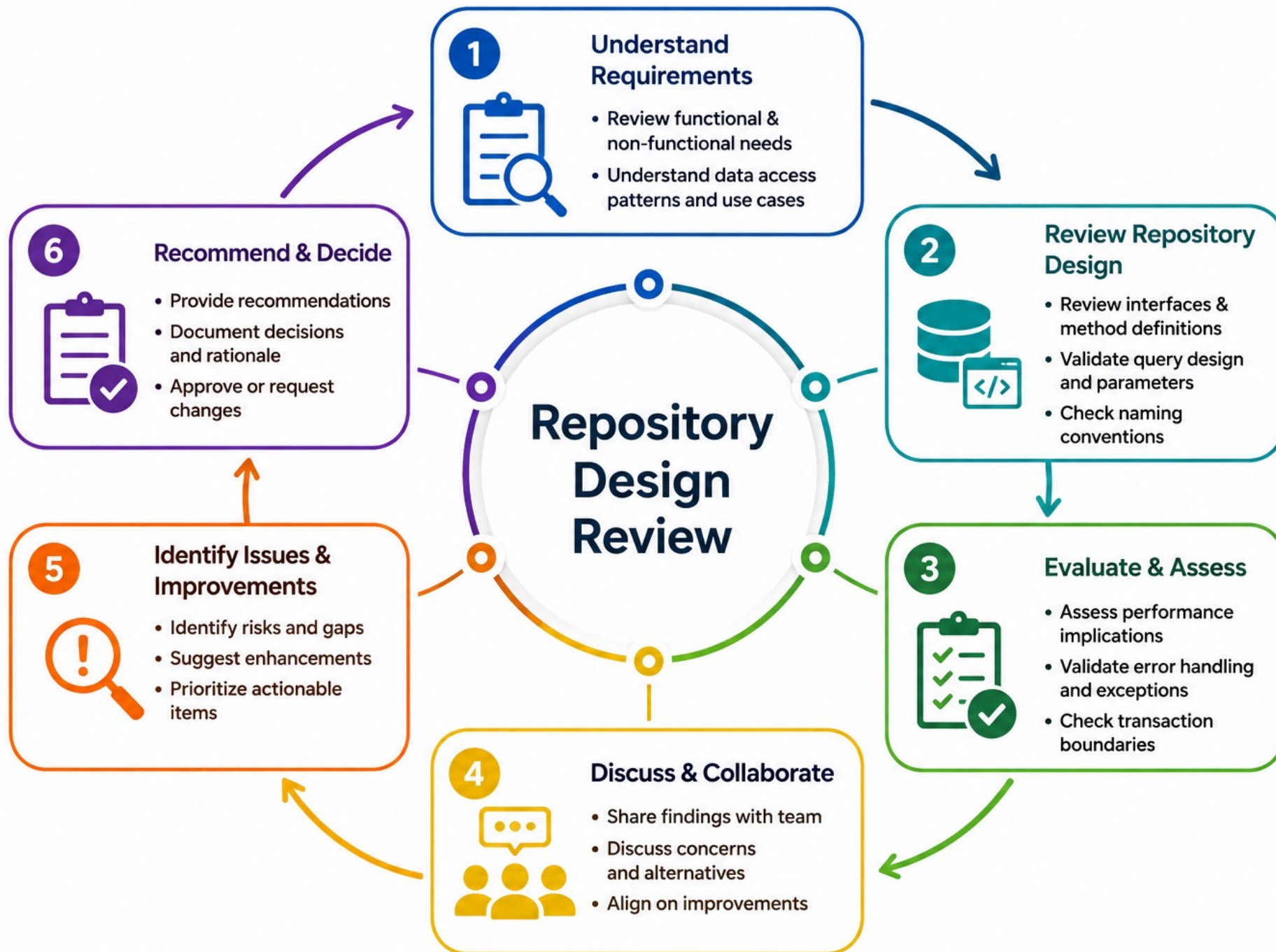
MIGRATION DISCIPLINE

- A Flyway migration for customer_interaction, PostgreSQL-compatible types.
- Document which profile runs against PostgreSQL versus H2.
- Inspect the generated SQL once, in a test or a log.
- Index the timeline-by-customer-and-time lookup.

Migration filename pattern

```
db/migration/V3__customer_interaction.sql
```

KEY TAKEAWAY A flaky seed with random emails is a named failure mode -- use the fixed fixture IDs, never random data, for deterministic repository tests.



CUSTOMER API DESIGN

The endpoint shape decided in Module 48 becomes real code here -- status codes, headers, and the customer resource path.

ENDPOINT SHAPE

POST /api/customers/{customerId}/interactions -> 201 + Location.

A GET timeline endpoint, if the CAP story requires it for Lab 50.

@PreAuthorize stubs now -- Lab 51 hardens fully.

The X-Correlation-ID header is read, never silently ignored.

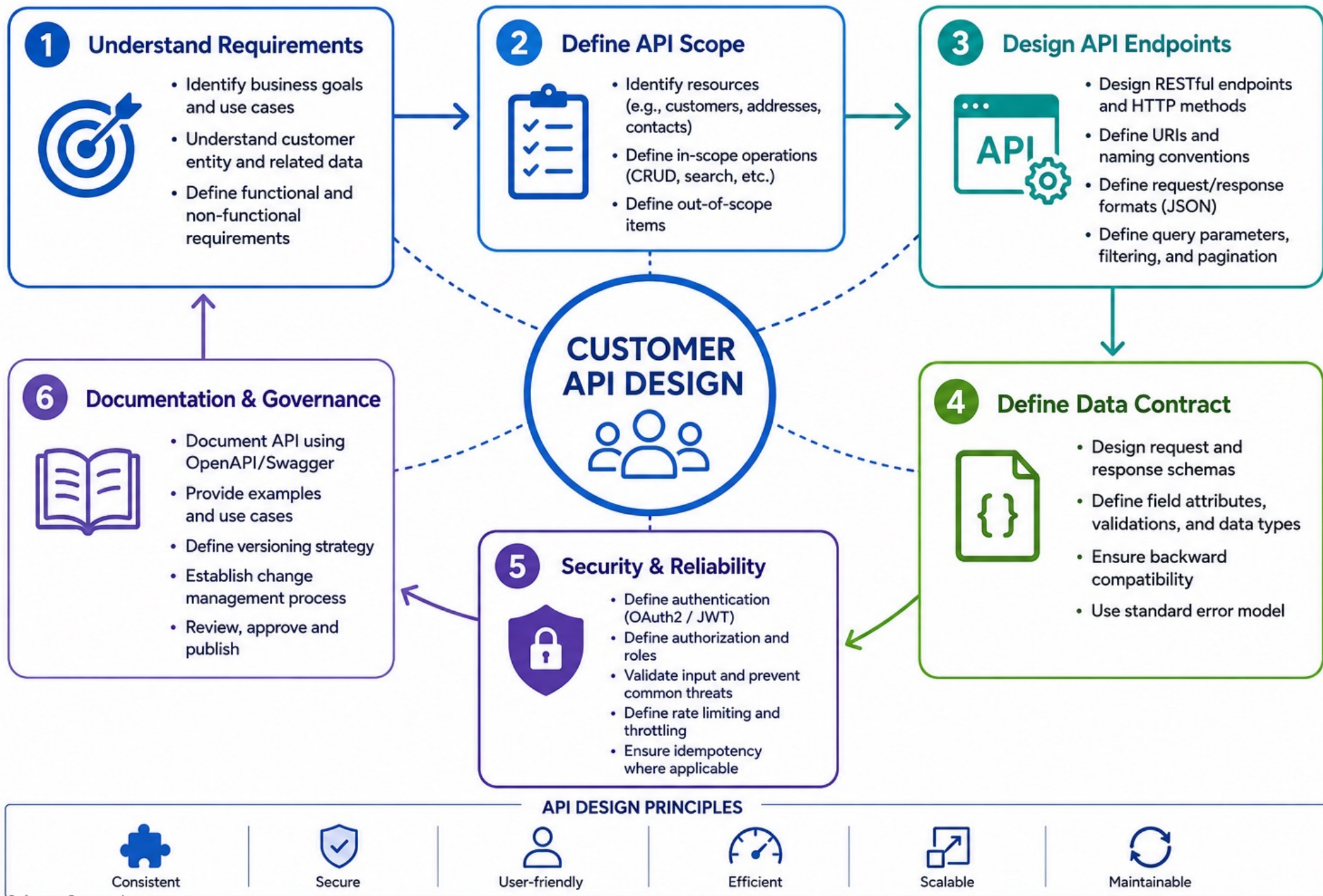
WHY THE SHAPE MATTERS

- 200 on create is a named failure to fix -- use 201.
- Stack traces in the response body block frontend integration.
- The path variable type must stay frozen for Lab 50's client.
- MockMvc snippets become the demo evidence, sanitized.

curl (worked example)

```
curl -i -X POST ../api/customers/$CUSTOMER_ID/interactions \
  -H 'X-Correlation-ID: lab-request-001' \
  -d '{"channel":"PHONE","summary":"..."}'
```

KEY TAKEAWAY 201 + a Location header, Problem Details for errors, and a stable path-variable type -- freeze the contract now, not after Lab 50 depends on it.



API CONTRACT DEFINITION

DTO records, not JPA entities, are the API contract -- keep persistence internal and clients typed.

DTO SHAPES

CreateInteractionRequest -- channel + summary, with Bean Validation.

InteractionResponse -- id, customerId, channel, summary, createdAt.

channel is an enum: PHONE, EMAIL, CHAT.

Entities never appear as a controller method's return type.

CONTRACT RULES

- Additive fields are safe; breaking changes need a version bump.
- A mapper converts entity to DTO and back, at the boundary.
- Field names get frozen in docs/backend-demo.md for Lab 50.
- No JPA annotation ever appears on a DTO record.

DTO records (excerpt)

```
record CreateInteractionRequest(  
    @NotBlank @Size(max=20) String channel,  
    @NotBlank @Size(max=1000) String summary) {}
```

KEY TAKEAWAY Shipping JPA entities as JSON couples clients to persistence and breaks Lab 50 typing -- DTO records are the only public contract.

API Contract Definition

1. API Overview



- Purpose: Manage customer data in the system
- Domain: Customer Management
- Base URL: <https://api.example.com/v1>
- Version: v1
- Owner: API Team
- Last Updated: 2026-05-08

2. Endpoints



Method	Endpoint	Description
GET	/customers	List all customers
GET	/customers/{id}	Get customer details
POST	/customers	Create a new customer
PUT	/customers/{id}	Update customer
DELETE	/customers/{id}	Delete customer

3. Request



Headers

Header	Description
Content-Type	application/json
Authorization	Bearer {token}
X-Correlation-ID	Unique request ID

Path Parameters

Parameter	Type	Required	Description
id	string	Yes	Customer ID

Request Body (POST / PUT)

```
{  "name": "John Doe",  "email": "john.doe@example.com",  "phone": "+1-987-654-3210",  "status": "ACTIVE"}
```

API CONTRACT

The agreement between providers and consumers on how the API works.

4. Response



Success Response (200 OK)

```
{  "id": "c12345",  "name": "John Doe",  "email": "john.doe@example.com",  "phone": "+1-987-654-3210",  "status": "ACTIVE",  "createdAt": "2026-05-08T10:30:00Z",  "updatedAt": "2026-05-08T10:30:00Z"}
```

Error Response (400 Bad Request)

```
{  "code": "VALIDATION_ERROR",  "message": "Email is invalid",  "details": [ { "field": "email", "issue": "Invalid format" } ]}
```

5. Data Model



Customer

Field	Type	Required	Description
id	string	Yes	Unique customer ID
name	string	Yes	Full name
email	string	Yes	Email address
phone	string	No	Phone number
status	string	Yes	Customer status
createdAt	string (date-time)	Yes	Creation timestamp
updatedAt	string (date-time)	Yes	Last update timestamp

6. Policies & Rules



- Authentication: OAuth2 / JWT Bearer Token
- Authorization: Role-based access control
- Rate Limiting: 1000 requests per minute
- Pagination: page, size, sort parameters
- Idempotency: Idempotency-Key header support
- Caching: Cache-Control max-age=60

7. Contract Format



OpenAPI (Swagger)



```
openapi: 3.0.3
info:
  title: Customer API
  version: '1.0.0'
servers:
  - url: https://api.example.com/v1
paths:
  /customers:
    get:
      summary: List customers
      ...
```

8. Governance



- Backward Compatibility: Maintain non-breaking changes
- Versioning Strategy: URI versioning (e.g., /v1/)
- Change Management: Review & approval process
- Deprecation Policy: Notify & sunset timeline
- Documentation: Keep contract up to date
- Monitoring & Analytics: Track usage & errors

REQUEST VALIDATION STRATEGY

Bean Validation on DTOs, enforced with @Valid at the boundary, rejects bad input before it ever reaches the service layer.

VALIDATION RULES

@NotBlank and @Size on both channel and summary.

channel is restricted to PHONE, EMAIL, or CHAT.

@Valid on the controller parameter triggers validation.

Invalid input never reaches the service or the database.

WHAT VALIDATION PROVES VS. JWT

- Validation proves shape and content; JWT (Lab 51) proves identity.
- 400 for invalid channel; 404 for CUS-9999 -- different failure classes.
- An empty summary must return 400 and persist nothing.
- Validation failures never publish a Kafka event.

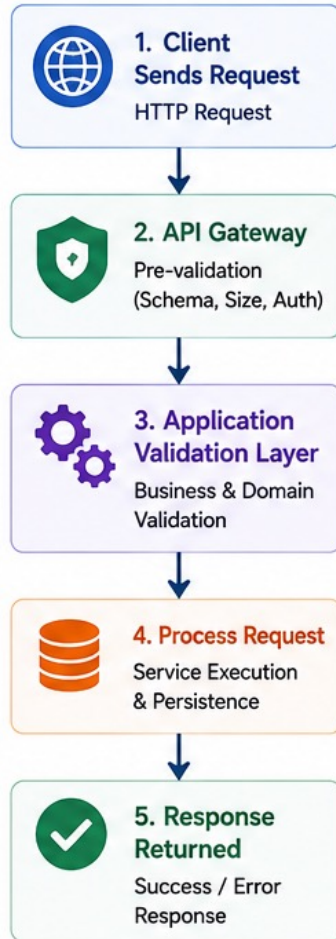
Expected result (MockMvc)

```
201 for Amina.    400 invalid channel.    404 CUS-9999.    Location header present.
```

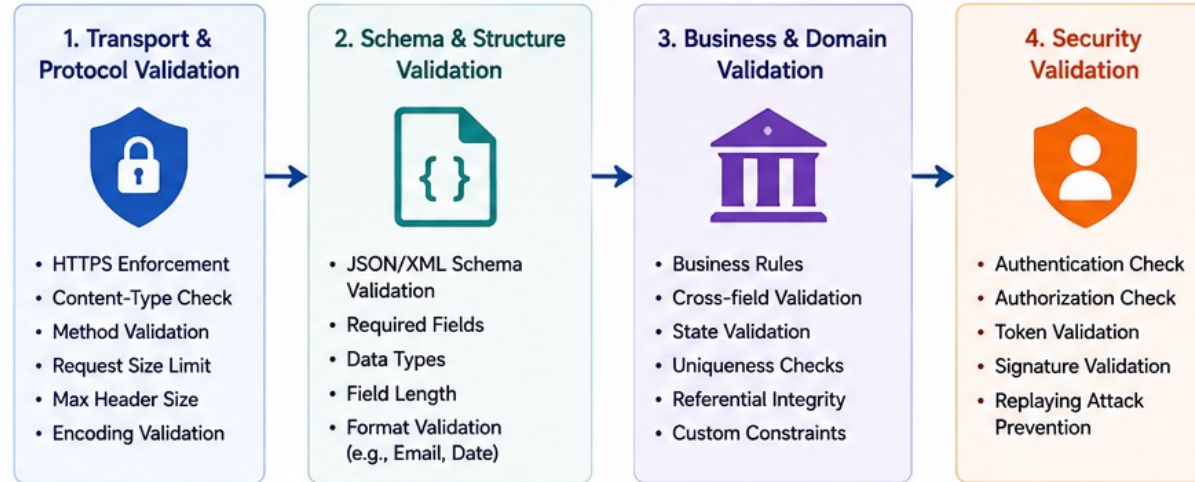
KEY TAKEAWAY Wrong status codes and opaque 500s block frontend integration and panel trust -- validate at the boundary, every time.

Request Validation Strategy

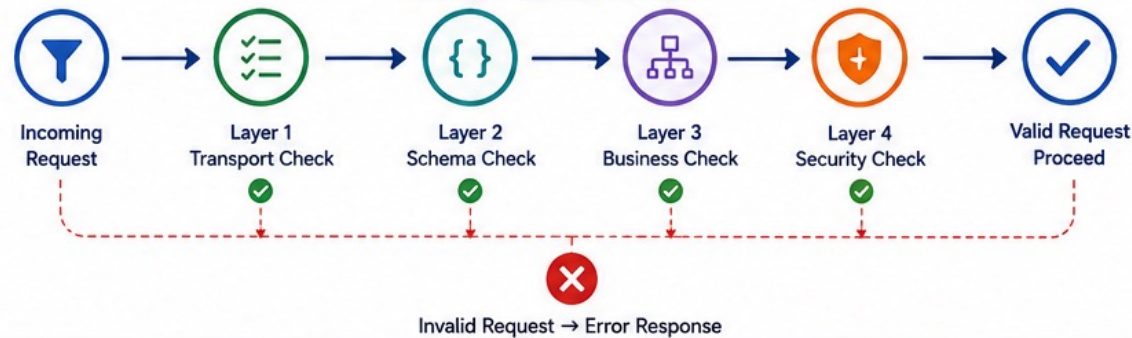
Flow of Request



Validation Layers



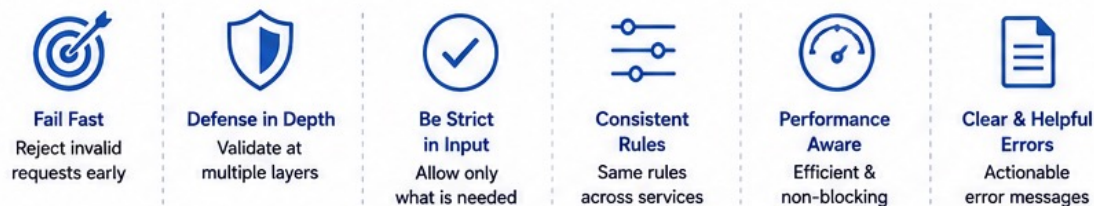
Validation Execution Flow



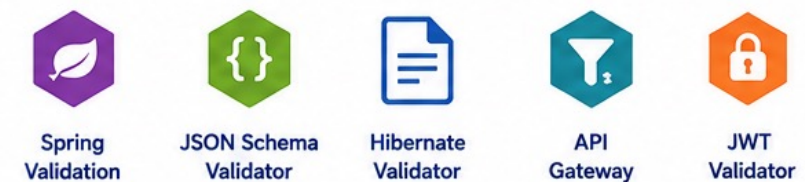
Error Handling Strategy

- 1. Consistent Error Response**
 - Standard Error Format
 - Error Code
 - Human Readable Message
 - Field-level Errors
- 2. HTTP Status Codes**
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 422 Unprocessable Entity
- 3. Field-Level Errors**
 - Return invalid fields
 - Provide reason
 - Support multiple errors
- 4. Logging & Monitoring**
 - Log validation failures
 - Track error trends
 - Alerts for suspicious activity

Validation Principles



Tools & Technologies



EXCEPTION HANDLING STRATEGY

A ProblemDetailsAdvice turns exceptions into a consistent, typed error contract instead of leaking stack traces.

PROBLEM DETAILS SHAPE

type, title, status, detail, instance -- on every error response.

A validation failure returns 400 with field-level detail.

CustomerNotFoundException returns 404, a bounded error.

Never leak a stack trace or an internal exception message.

MAPPING EXCEPTIONS

- @ControllerAdvice centralizes exception-to-response mapping.
- Domain exceptions are typed, not a generic RuntimeException.
- Security auto-config surprises get their own test security config.
- Errors are logged with the correlation id, never note content.

Problem Details response (excerpt)

```
{"type":"about:blank","title":"Bad Request","status":400,  
  "detail":"Validation failed","instance":"/api/customers/.../interactions"}
```

KEY TAKEAWAY A consistent Problem Details contract is what lets Lab 50's UI render real error messages instead of a generic failure screen.

Exception Handling Strategy

Goal: Handle exceptions consistently, provide meaningful feedback, and ensure system stability.



EVENT-DRIVEN ARCHITECTURE REVIEW

A quick review of event-driven concepts this module puts into practice for the customer interaction slice.

Term	Meaning
Producer	Publishes an event after a successful, committed action.
Consumer	Subscribes to a topic and reacts -- must tolerate duplicates.
Partition Key	Determines ordering -- customer id keeps one customer's events in order.
Dead Letter Topic (DLT)	Where poison messages land after bounded retries are exhausted.

WHY EVENT-DRIVEN HERE



Decouples Side Effects

Notification/worker consumers react without the API waiting on them.



At-Least-Once Delivery

Duplicate events are normal -- design consumers to be idempotent.



Ordering by Customer

Partition key = customer id keeps one customer's timeline events ordered.



Lag Is Observable

A lag alert and a DLT turn silent data loss into a visible, testable failure.

KEY TAKEAWAY Unversioned, fire-and-forget events become silent data loss under consumer lag -- version, key, and observe every event.

Event-Driven Architecture Review

Decoupled • Scalable • Responsive • Resilient

1. Overview

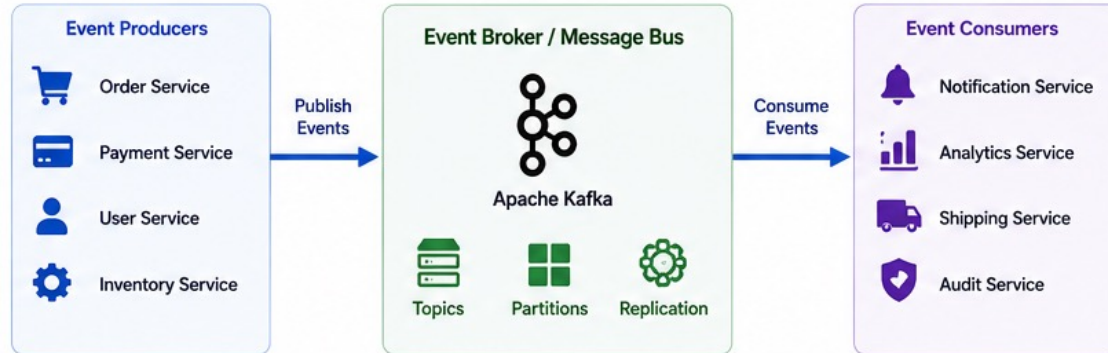


Event-Driven Architecture (EDA) enables systems to communicate by producing, publishing and consuming events.

Key Benefits

- ✓ Loose Coupling
- ✓ Scalability
- ✓ Real-time Responsiveness
- ✓ Resilience
- ✓ Extensibility

2. EDA Flow



3. Design Principles



Events are the Source of Truth
State changes are captured as immutable events.



Loose Coupling
Producers and consumers are independent.



Asynchronous Communication
Improves throughput and system responsiveness.



Scalability
Scale producers and consumers independently.



Resilience
Failures are isolated and recoverable.

4. Event Lifecycle

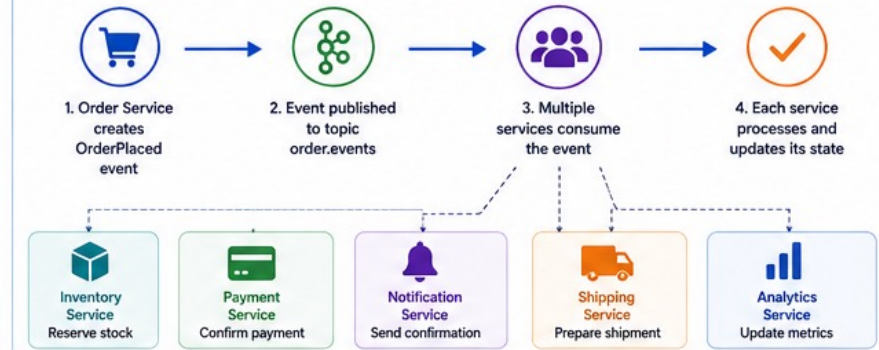


- 1. Create**
Event is created when a state change occurs
- 2. Publish**
Event is published to the broker (topic)
- 3. Store**
Event is persisted in partitions with replication
- 4. Consume**
Consumers read events at their own pace
- 5. Process**
Event is processed and triggers action
- 6. Acknowledge**
Offset is committed and event is considered done

5. Core Components

Component	Description
Event Producers	Generate events when business state changes.
Event Broker (Message Bus)	Receives, stores, and distributes events to consumers.
Event Consumers	Subscribe to events and perform processing.
Event Schema (Contract)	Defines the structure and version of events.
Schema Registry	Stores and manages event schemas and versions.
Monitoring & DLQ	Monitors health and handles failed events.

6. Example: Order Placed Flow



7. Event Design Best Practices



8. Common Patterns



9. Considerations & Trade-offs

- ⚠ Eventual consistency vs strong consistency
- ⚠ Complexity in debugging and tracing
- ⚠ Need for idempotent consumers
- ⚠ Schema evolution and versioning
- ⚠ Monitoring, replay and data retention

10. Tools & Technologies



CUSTOMER EVENT DESIGN

CustomerInteractionRecordedV1 is the versioned event contract Lab 49 publishes and later labs will consume.

EVENT FIELDS

eventId, eventType, eventVersion.

occurredAt, correlationId, actor.

customerId, interactionId, channel.

Topic: crm.customer.interactions.v1.

DESIGN RULES

- Partition key = customer id, for ordered per-customer delivery.
- Publish only after the documented consistency strategy (Module 48 ADR).
- Additive fields are safe; breaking changes bump the version.
- Never publish on a validation failure.

CustomerInteractionRecordedV1 (record)

```
record CustomerInteractionRecordedV1(  
  UUID eventId, String eventType, int eventVersion,  
  Instant occurredAt, String correlationId, String actor,  
  UUID customerId, UUID interactionId, String channel) {}
```

KEY TAKEAWAY One event for one successful create -- the payload must include lab-request-001 and Amina's customer id, every time.

Customer Event Design

1. Purpose



Define consistent, meaningful events to capture customer state changes and drive business processes.

Key Goals

- ✓ Capture important customer lifecycle changes
- ✓ Enable real-time integration and automation
- ✓ Ensure events are consistent, versioned, and reliable
- ✓ Support analytics and auditing

2. Customer Events Overview



3. Design Principles



Meaningful
Represent business-significant changes



Consistent
Use consistent naming, format and metadata



Independent
Events are self-contained and loosely coupled



Reliable
Delivered at least once with eventual consistency



Discoverable
Well documented and versioned for consumers

4. Customer Event Catalog

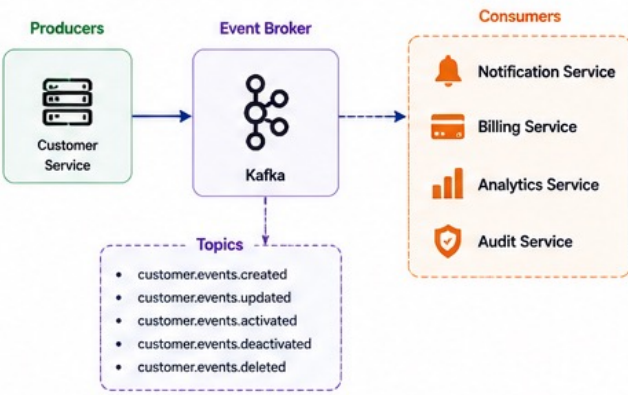
Event Name	Description	Trigger	Importance
CustomerCreated	Emitted when a new customer is created	Customer registration completed	High
CustomerUpdated	Emitted when customer profile is updated	Profile information changed	Medium
CustomerActivated	Emitted when customer account is activated	Account activation successful	High
CustomerDeactivated	Emitted when customer account is deactivated	Account deactivated by user or system	Medium
CustomerDeleted	Emitted when customer is permanently deleted	Customer deletion completed	High

5. Event Structure (Example: CustomerCreated)

```
{  "eventId": "c8f4a3e2-9d77-4b58-8e2b-6f1b3d5a9c01",  "eventType": "CustomerCreated",  "eventVersion": "1.0",  "occurredAt": "2025-05-08T10:15:30Z",  "source": "CustomerService",  "correlationId": "9b7d8c6a-1234-4e8f-91a3-4a2d6e7f1234",  "data": {    "customerId": "CUST-10001",    "email": "john.doe@example.com",    "firstName": "John",    "lastName": "Doe",    "phone": "+1-555-123-4567",    "createdAt": "2025-05-08T10:15:30Z"  },  "metadata": {    "schemaVersion": "1.0",    "tenantId": "TENANT-01"  }}
```

Field	Description
eventId	Unique identifier of the event
eventType	Type of the event
eventVersion	Version of the event schema
occurredAt	UTC timestamp when event occurred
source	Service that published the event
correlationId	For tracking across services
data	Event payload
metadata	Additional metadata about the event

6. Event Routing & Topics



7. Schema Evolution & Versioning



- Initial schema
- required fields



- New optional fields
- backward compatible



- Breaking changes
- new version

Guidelines

- ✓ Use backward-compatible changes
- ✓ Add new fields as optional
- ✓ Deprecate before removal
- ✓ Version in event metadata

8. Governance & Quality



Ownership
Clear owner for each event



Documentation
Well documented in event catalog



Validation
Schema validated at producer



Testing
Contract tests for consumers



Monitoring
Track delivery and error rates



Retention
Retain events as per policy

9. Best Practices

- ✓ Keep events small and focused
- ✓ Use past-tense, domain-driven names
- ✓ Make events immutable
- ✓ Include correlation and causation IDs
- ✓ Ensure idempotent consumers
- ✓ Monitor and alert on failures
- ✓ Review and evolve events continuously

TRY IT YOURSELF -- EXERCISE 3: SKETCH EVENT CONTRACT

Reinforces: drafting a versioned event payload outline for the slice, with an example JSON and a compatibility note.

STEPS (notes/lab49-event-contract.md)

Step	What To Do
1 -- Fields	eventType, eventVersion, customerId, occurredAt, correlationId, payload{}
2 -- Check the Reference	Consumers must be idempotent -- duplicates are normal, not an edge case.
3 -- Example	Write one example JSON for CUS-1001 with correlation lab-request-001 (synthetic).
4 -- Compatibility	One sentence on how you will handle a v2 field addition later.

Event sketch

```
{
  "eventType": "CustomerInteractionRecorded",
  "eventVersion": 1,
  "customerId": "CUS-1001",
  "correlationId": "lab-request-001"
}
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-event-contract.md (workspace: examples/module-49-exercises).

PRODUCER / CONSUMER IMPLEMENTATION

InteractionEventPublisher sends after commit; a resilient consumer dedupes, retries with a bound, and routes poison messages to a DLT.

PRODUCER (InteractionEventPublisher)

Publishes only after the service's transaction commits.

Partition key = customer id.

Injected as a collaborator -- never a static Kafka client in the domain.

Unit-tested with a fake publisher recording the payload.

CONSUMER (resilient)

- Validates eventVersion; rejects incompatible events.
- Deduplicates on eventId -- duplicate delivery is a no-op.
- Bounded retries, then routes to a dead letter topic.
- Logs the correlation id -- never the note body.

Consumer dedupe check (concept)

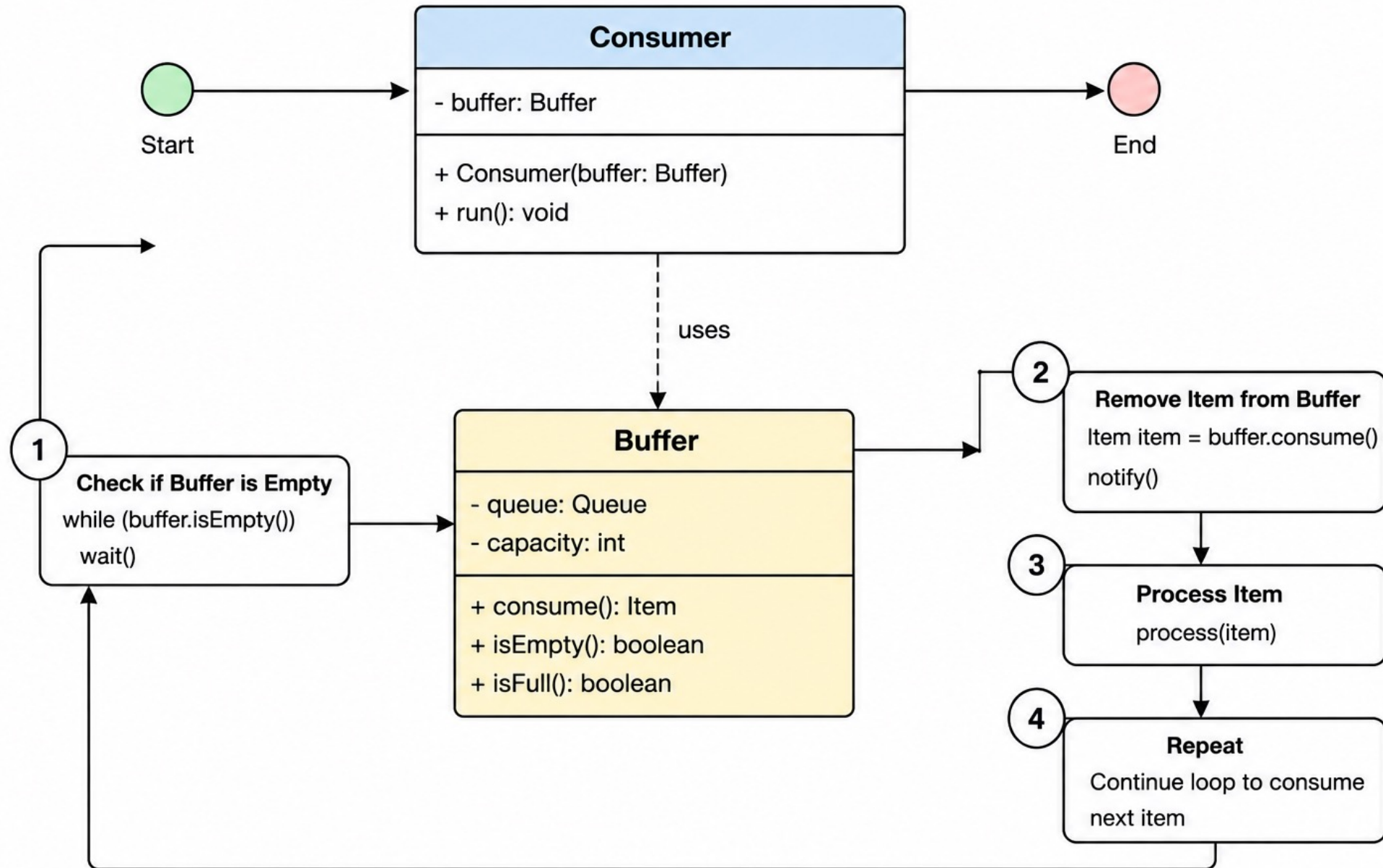
```
if (seen(event.eventId())) return; // no-op on duplicate
process(event); markSeen(event.eventId());
```

KEY TAKEAWAY At-least-once delivery without dedupe double-sends side effects and fails the defense -- bound retries, dedupe, and use a DLT.

2



Consumer Implementation

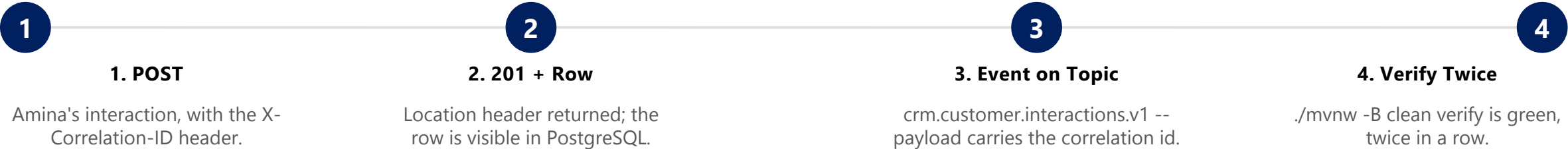


EVENT PROCESSING WORKFLOW

One request, four checkpoints -- from an HTTP POST to a durable event a consumer can safely process twice.

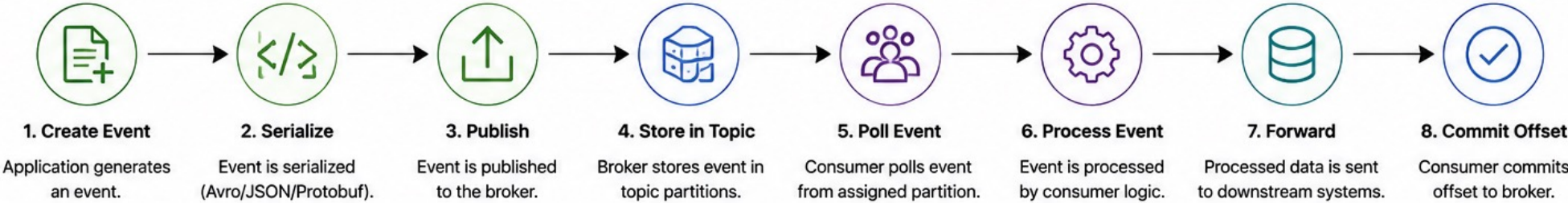
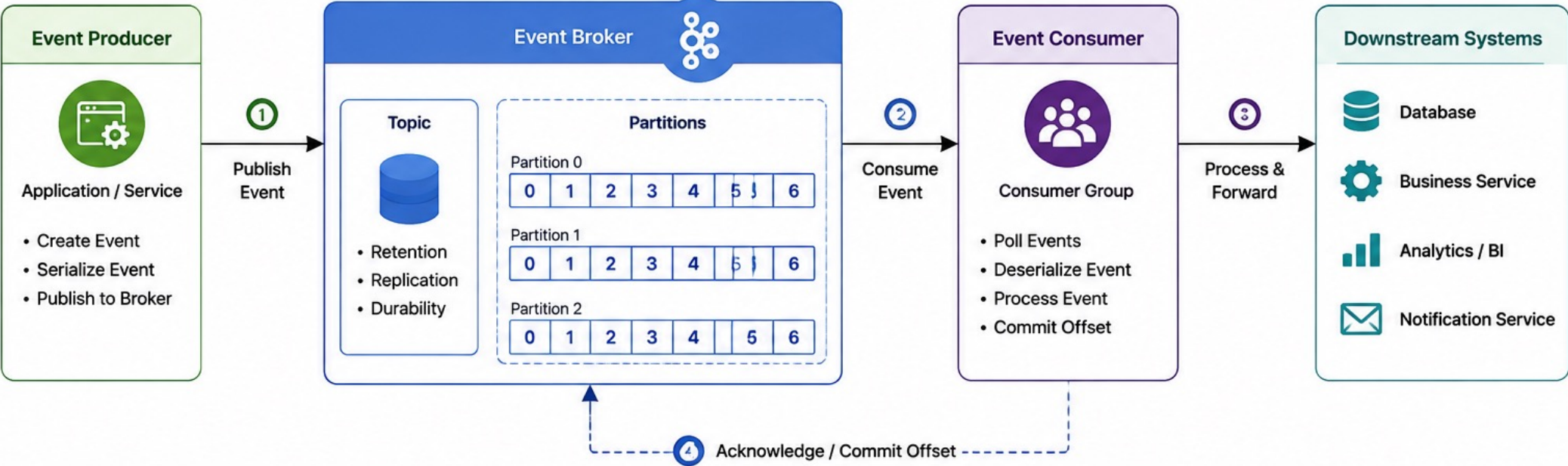
Stage	What Happens	Verified By
1. Validate	Bean Validation rejects bad input before persistence.	MockMvc 400 test
2. Persist	InteractionService saves inside @Transactional.	Repository test, seeded Amina
3. Publish	Event sent after commit, keyed by customer id.	Kafka IT, payload assertion
4. Consume	Consumer dedupes on eventId, processes once.	IT: duplicate delivery is a no-op

END-TO-END TRACE (lab-request-001)



KEY TAKEAWAY A single request traced from API to database to event, twice reproducibly, is stronger demo evidence than five confident claims.

Event Processing Workflow



★ **Key Benefits**

✓ Decoupled Systems

✓ Scalable & Parallel Processing

✓ Reliable & Durable

✓ Fault Tolerant

✓ Real-time Data Flow

JWT AUTHENTICATION REVIEW

A review of the JWT/OIDC resource-server pattern decided in Module 48's ADR-004 -- Lab 49 stubs it, Lab 51 hardens it.

Concept	Meaning
Resource Server	Validates JWTs on incoming requests; does not issue tokens itself.
Bearer Token	Authorization: Bearer <token> header on every authenticated request.
Claims	Role and identity information inside the JWT -- validated, not trusted blindly.
Test Security Config	A minimal security setup so unit/MVC tests are not blocked by full auth.

JWT REVIEW HABITS

**Stub Now, Harden Later**

@PreAuthorize stubs land in Lab 49; Lab 51 enforces deny-by-default fully.

**Bearer Token Always**

Every authenticated curl and test carries Authorization: Bearer \$TOKEN.

**Test Security Config**

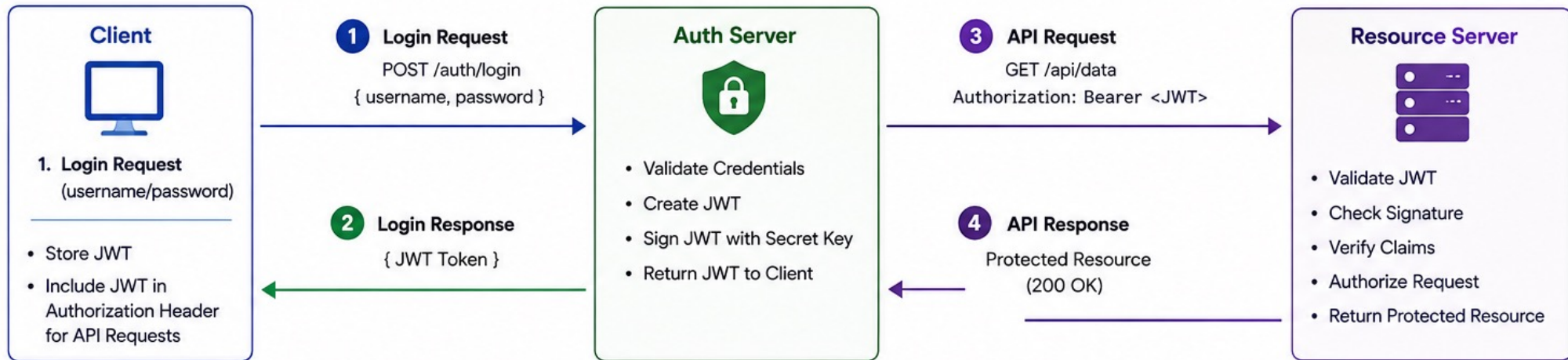
Add a minimal security config so 401/403 auto-config surprises do not block tests.

**Never Trust Claims Blindly**

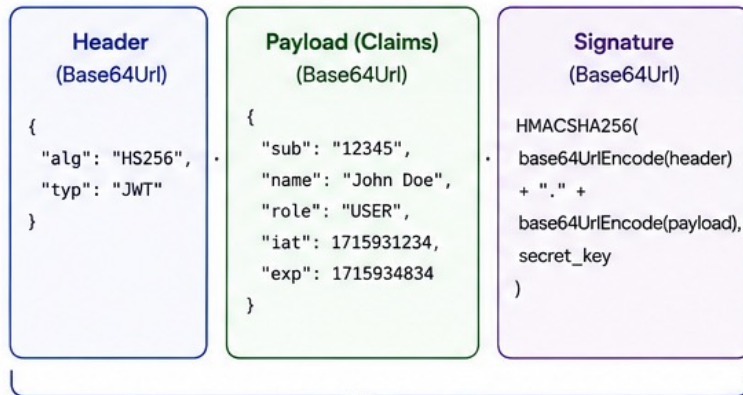
Claims still get validated -- a JWT is a claim, not automatic truth.

KEY TAKEAWAY 401/403 surprises from security auto-config are a named troubleshooting item -- add a test security config early, not mid-debug.

JWT Authentication Review

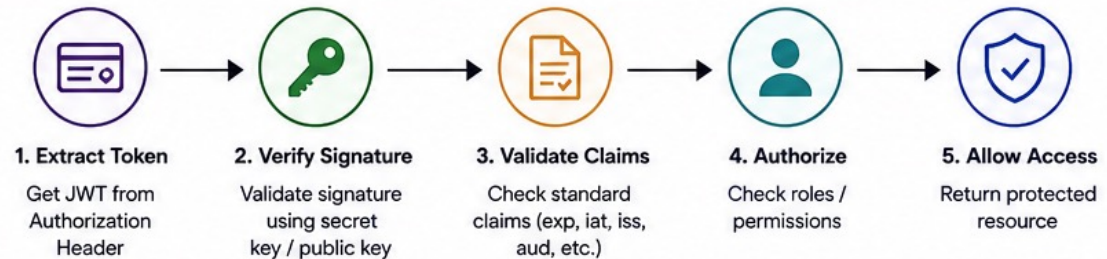


JWT Structure



JWT Token

JWT Validation Process (Resource Server)



Common Claims

sub	iss	aud	exp	iat	nbf	jti
Subject	Issuer	Audience	Expiration Time	Issued At	Not Before	JWT ID

Key Benefits



Stateless
No server-side session storage



Scalable
Works across distributed systems



Secure
Digitally signed and tamper-proof



Standard
Open standard (RFC 7519)



Interoperable
Supported by many platforms

ROLE-BASED ACCESS CONTROL

AGENT and MANAGER are the minimum roles Module 48 planned -- RBAC decisions get annotated now and enforced fully in Lab 51.

ROLE MODEL

AGENT: create and view interactions.

MANAGER: everything AGENT can do, plus oversight views.

@PreAuthorize("hasAnyRole('AGENT','MANAGER')") on the create endpoint.

Roles, never per-person special cases.

RBAC TESTING

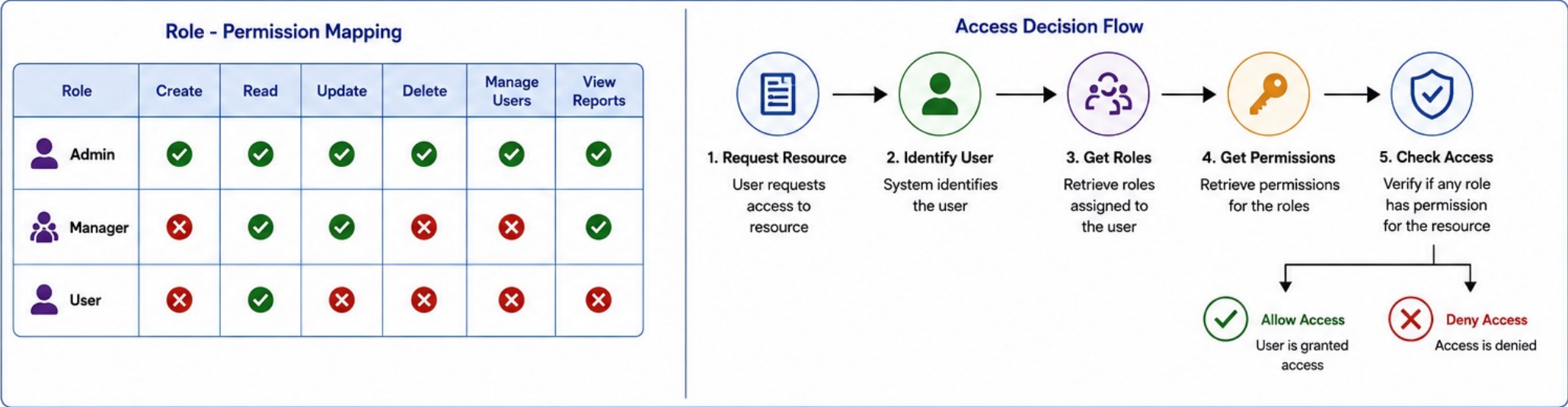
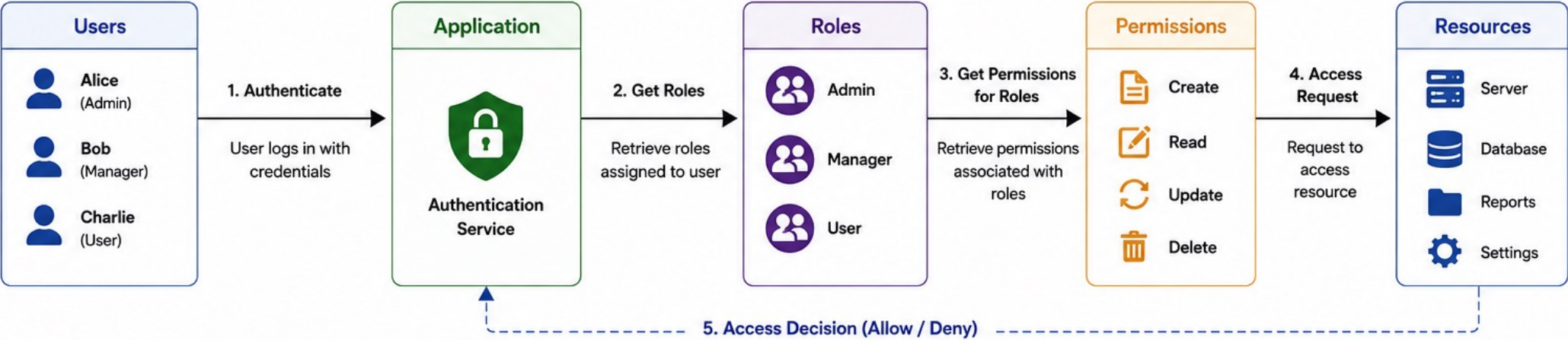
- Negative test: wrong role -> 403.
- Negative test: no token -> 401.
- Role checks live in annotations, not scattered if-statements.
- RBAC decisions match the Module 48 security ADR.

Endpoint annotation (excerpt)

```
@PreAuthorize("hasAnyRole('AGENT','MANAGER')")  
ResponseEntity<InteractionResponse> create(...)
```

KEY TAKEAWAY Plan roles, not people -- a role-based system scales across a team; a person-based one does not.

Role-Based Access Control



Key Benefits

- Enhanced Security**: Access is granted only based on roles and permissions.
- Simplified Management**: Manage permissions by assigning roles instead of individual users.
- Scalability**: Easily scale by adding new roles and permissions.
- Consistency**: Enforces consistent access control policies.
- Audit & Compliance**: Easier to audit and meet compliance requirements.

ENDPOINT PROTECTION

*Every /api/** endpoint is protected by default -- an open endpoint is a planning failure, not a Lab 51 surprise.*

PROTECTION RULES

Unauthenticated /api/** -> 401 (the Module 48 security NFR).

Wrong role -> 403, never a silent 200.

Path variables validated -- no type confusion with Lab 50's client.

The correlation header is read safely, never trusted for authorization.

WHAT'S STUBBED VS. HARDENED

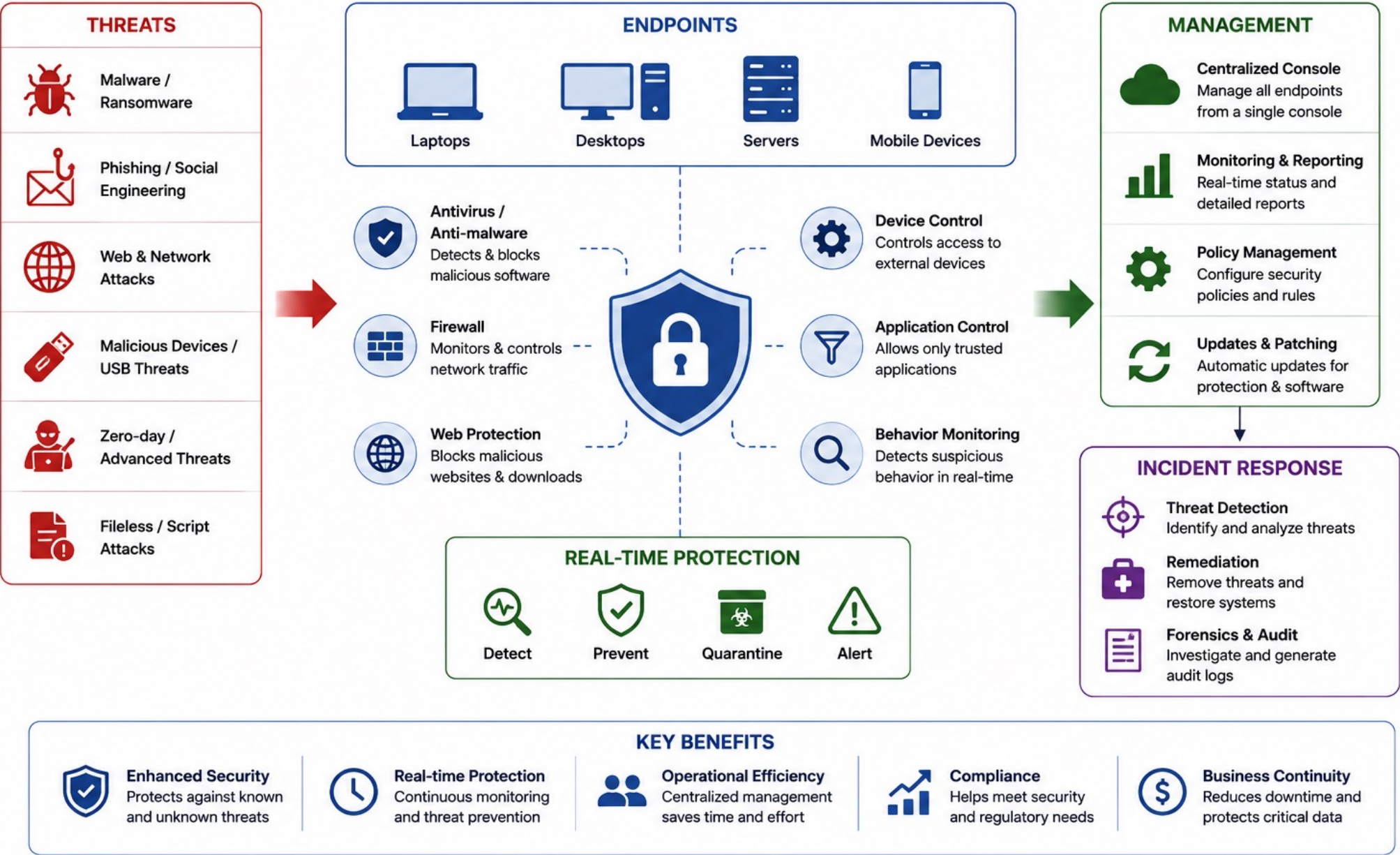
- Lab 49: @PreAuthorize stubs and role annotations.
- Lab 51: full JWT deny-by-default, scanners, and negative tests in CI.
- Document which is stubbed vs. complete in backend-demo.md.
- Never claim endpoint protection is 'done' before Lab 51.

Security NFR (from Module 48)

```
"Unauthenticated /api/** -> 401; wrong role -> 403" -- verified by security tests.
```

KEY TAKEAWAY Document honestly what's stubbed versus hardened -- claiming full protection before Lab 51 is a documentation integrity issue.

Endpoint Protection



UNIT TESTING STRATEGY

Unit tests isolate the service layer with a fake publisher -- no Spring context, no network, fast feedback.

WHAT UNIT TESTS COVER

InteractionService happy path -- customer found, interaction saved.

The not-found path -- CUS-9999 throws, publish is never called.

A fake publisher records the exact payload for assertions.

No Spring context needed -- keeps the test suite fast.

UNIT TEST DISCIPLINE

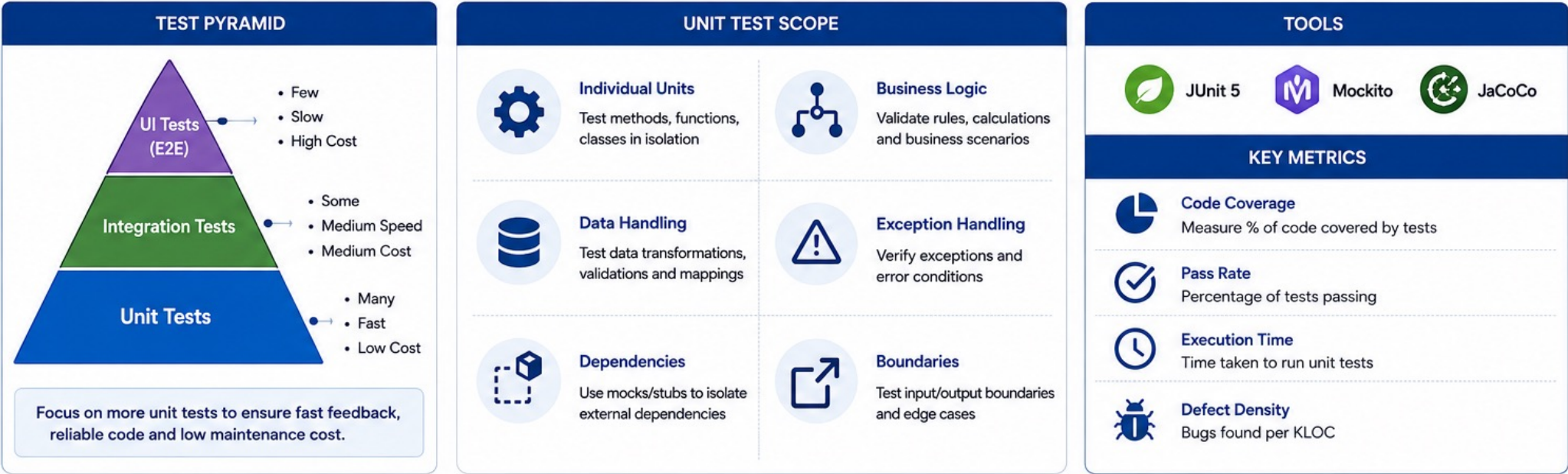
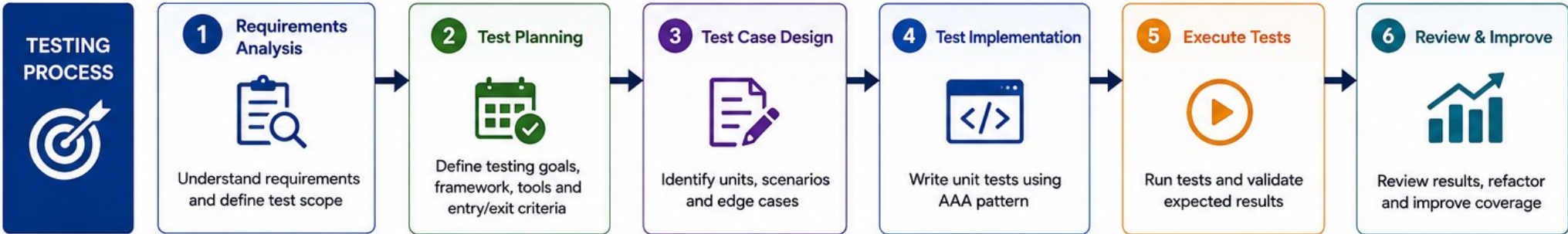
- Assert a domain outcome, not just 'no exception was thrown'.
- Reject false-confidence tests that never assert anything real.
- Name tests by behavior: create_savesAndPublishes_onValidRequest.
- Fast tests run on every save; slower ones run before every commit.

Service unit test (concept)

```
assertThat(fakePublisher.lastEvent().customerId()).isEqualTo(...)  
assertThat(fakePublisher.published()).isFalse(); // not-found path
```

KEY TAKEAWAY A test that never asserts a domain outcome is false confidence -- this module explicitly rejects tests that only check 'no exception thrown'.

Unit Testing Strategy



INTEGRATION TESTING STRATEGY

MockMvc, JPA, and Kafka integration tests prove the layers work together, not just in isolation.

INTEGRATION LAYERS

MockMvc: 201 for Amina, 400 invalid channel, 404 CUS-9999.

JPA/repository test: a round-trip save against a real or test database.

Kafka IT: one event on the topic per successful create.

Prefer Testcontainers or an approved Compose setup for realism.

KEEPING ITs RELIABLE

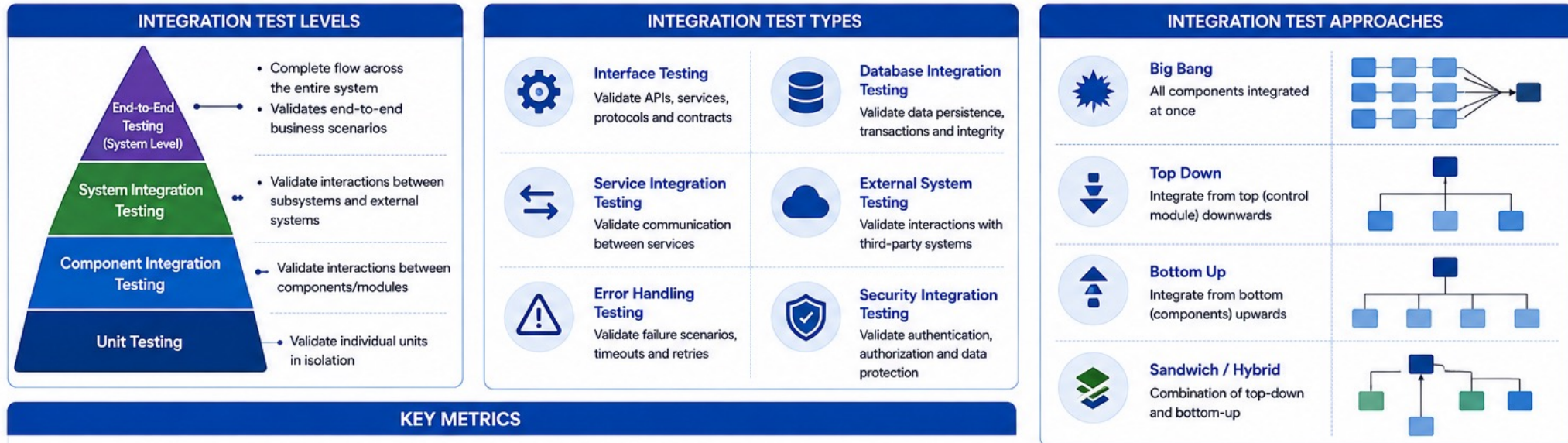
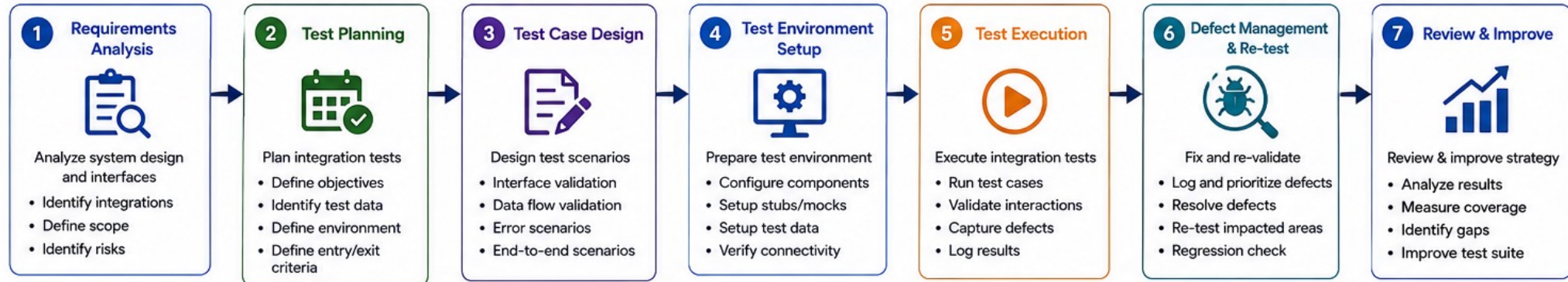
- Unique event IDs and awaitility -- avoid flaky timing assumptions.
- Fix shared topic pollution between test runs.
- Two consecutive mvn clean verify runs must match.
- Document which profile runs against PostgreSQL vs. H2.

Verify command

```
./mvnw -B clean verify
```

KEY TAKEAWAY Untested messaging is a live-demo mystery failure in Lab 52 -- integration tests are what turn 'it worked once' into evidence.

Integration Testing Strategy



TRY IT YOURSELF -- EXERCISE 4: FILL TEST MATRIX TODOS

Reinforces: completing a happy/failure test matrix for Lab 49, with at least one concrete happy and one failure case.

STEPS (notes/lab49-test-matrix-todos.md)

Step	What To Do
1 -- Template	Unit rule, validation failure, integration happy, Kafka publish, idempotent consume, authz negative.
2 -- Minimum Bar	At least one happy and one failure path must be concrete, not a placeholder.
3 -- Evidence	Name Surefire report path placeholders for the demo notes.
4 -- Scope	Plan only -- writing the actual tests happens during Lab 49.

Test matrix row

Unit test service rule: _____ Validation failure case: _____
Integration API happy: _____ Kafka publish assertion: _____

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-test-matrix-todos.md (workspace: examples/module-49-exercises).

TEST COVERAGE REVIEW

Coverage is a health signal, not the goal -- a covered line with no domain assertion is still false confidence.

WHAT TO COVER

The happy path: create for CUS-1001.

The not-found path: CUS-9999.

Validation failure: empty summary, an invalid channel.

Duplicate event consumption: no double side effect.

COVERAGE PITFALLS

- High line coverage with weak assertions is still a weak suite.
- A test suite with no negative path leaves failure handling ungraded.
- Coverage reports are evidence, not a substitute for review.
- Cross-check coverage against CAP-12's acceptance criteria.

Coverage-to-AC cross-check

```
AC1 (201 + id) -> InteractionControllerTest.create_returns201
AC4 (invalid notes) -> InteractionControllerTest.create_rejectsBlankSummary
```

KEY TAKEAWAY Cross-check test coverage against CAP-12's numbered acceptance criteria -- a passing suite that skips an AC is not actually done.

CODE REVIEW PROCESS

The same peer-review discipline from Module 47 applies to backend code -- cite the line, propose the rewrite, record the disposition.

REVIEW FOCUS FOR THIS SLICE

Are entities kept out of the public API?

Does the service own the transaction and publish boundary?

Are the negative paths (400/404) actually tested?

Is the correlation id threaded through logs and the event?

PR DESCRIPTION FOR CAP-12

- Why: the customer or operational problem being solved.
- What changed: backend, messaging, and config, listed separately.
- Verification: a pass/fail table -- tests, security checks.
- Risk and rollback: the exact rollback command or digest.

Reviewer focus questions (example)

```
"Does the consumer dedupe on eventId?"  
"Is entities/Interaction.java ever returned from a controller?"
```

KEY TAKEAWAY 'Looks good' is not acceptable feedback here either -- cite the file and line, and propose the concrete rewrite.

BACKEND QUALITY CHECKLIST

Before calling CAP-12 done, run this checklist -- it is the definition of a demoable, defensible backend slice.



API Evidence

201 for Amina, 400 invalid, 404 CUS-9999, and the Location header present.



Persistence Proof

Migration applied; the row exists; the generated SQL inspected once.



Event Proof

One event per create, versioned, correlation id present, keyed by customer.



Tests + Docs

Unit/MVC/JPA/Kafka tests green twice; backend-demo.md lets a peer reproduce it.

KEY TAKEAWAY No merge of the interaction slice without API evidence, persistence proof, versioned event proof, automated tests, and a documented failure path.

TRY IT YOURSELF -- EXERCISE 5: BACKEND SLICE DOD

Reinforces: writing a Definition of Done checklist for Lab 49, requiring one failing-path proof, not just a green demo.

STEPS (notes/lab49-definition-of-done.md)

Step	What To Do
1 -- Items	API works, migration present, event versioned, tests green, demo notes, no secrets.
2 -- Check the Reference	A green demo alone is not enough -- a peer must reproduce it from the docs.
3 -- Negative Proof	Require one failing-path proof in the DoD -- not just the happy path.
4 -- Scope	DoD checklist only -- writing the slice itself happens during Lab 49.

DoD checklist skeleton

```
[ ] API 201/400/404 evidenced    [ ] Migration applied
[ ] Event on topic, versioned    [ ] Tests green x2    [ ] No secrets
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-definition-of-done.md (workspace: examples/module-49-exercises). Final gate before Lab 49.

TRY IT YOURSELF -- EXERCISE 6: OUTLINE BACKEND DEMO NOTES

Reinforces: outlining docs/backend-demo.md so a peer can reproduce the slice from commands alone.

STEPS (notes/lab49-backend-demo-outline.md)

Step	What To Do
1 -- Headings	Prereqs, start commands, curl/HTTPIe samples, expected JSON, test commands, known limits.
2 -- Samples	Include placeholder curls using CUS-1001 and the correlation header.
3 -- Limits	Document what is stubbed vs. complete for the day.
4 -- Scope	Outline warmup for Lab 49 -- the full runbook is written during the lab.

backend-demo.md skeleton

```
## Prerequisites  ## Seed customers  ## Happy path curl
## Negative path curl  ## SQL check  ## Kafka check  ## Test commands
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-backend-demo-outline.md (workspace: examples/module-49-exercises).

LAB OVERVIEW -- NORTHSTAR CRM INTERACTION SLICE

Implement the CAP-12 vertical slice: validated APIs, transaction-safe persistence, versioned events, resilient consumption, and automated tests.

BUSINESS SCENARIO

- Service agents must record interactions that persist reliably, return a clear API contract, publish a traceable event, and tolerate duplicate or failed consumption. A green demo without tests, correlation IDs, or failure-path evidence does not pass capstone quality.
- You own the backend gate using Module 48's CAP-12 acceptance criteria -- API evidence, persistence proof, versioned event proof, automated tests, and a documented failure path.
- A peer must be able to reproduce every result from docs/backend-demo.md alone, without verbal coaching.

LEARNING OBJECTIVES

- Implement layered Spring Boot features for the CAP-12 slice.
- Keep JPA entities out of external contracts; validate DTOs.
- Publish versioned Kafka events with correlation and actor metadata.
- Consume idempotently with bounded retries and a dead letter topic.
- Test HTTP, persistence, and messaging with failure paths.
- Document a reproducible backend demo with CUS-1001 and lab-request-001.

WHAT YOU BUILD

- customer-management-platform/backend/: InteractionController, InteractionService, InteractionRepository, CustomerInteractionRecordedV1, InteractionEventPublisher, a migration, and tests, plus docs/backend-demo.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina), CUS-1002 (Ravi), and CUS-9999 (not-found) are synthetic -- never real customer PII. Duration: about 45 minutes with starters, up to 6-8 hours full path. Advanced capstone difficulty.

LAB TASKS AND DELIVERABLES

lab49-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Select the vertical slice: CAP-12, with a written Definition of Done.
2	Create domain and DTOs: Bean Validation, entities kept internal.
3	Implement persistence: a Flyway migration, repository, seeded fixtures.
4	Build the application service: @Transactional, not-found handling, publish.
5	Expose the REST endpoint: 201 + Location, Problem Details for errors.
6	Publish the Kafka event: versioned, keyed by customer id.
7	Implement a resilient consumer: dedupe, bounded retries, a DLT.
8	Test the complete slice: unit, MockMvc, JPA, Kafka IT, and failure paths.

EXPECTED DELIVERABLES

- Backend source changes for the interaction vertical slice
- Database migration for interaction persistence
- Versioned event contract (CustomerInteractionRecordedV1)
- Unit and integration tests (HTTP, persistence, messaging)
- docs/backend-demo.md reproduction runbook

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 *
Integration/Config 15 * Failure Handling 15 * Verification
10 * Security/Production Awareness 10 * Docs 10

KEY TAKEAWAY Happy-path-only without negatives loses failure marks. Publishing on validation failure is an honor violation. A demo without lab-request-001 correlation is incomplete evidence.

MODULE 49 SUMMARY

In this module, we implemented CAP-12 end to end: a validated API, transactional persistence, a versioned Kafka event, and tests that prove it.

KEY CONCEPTS COVERED



Layered Service

Controller, service, repository -- each with one job, entities never public.



DTO Contracts

Bean Validation at the boundary; entities stay internal.



Versioned Events

CustomerInteractionRecordedV1 on `crm.customer.interactions.v1`, keyed by customer.



Idempotent Consumers

Dedupe on eventId, bounded retries, and a dead letter topic.



Tests as Evidence

Unit, MockMvc, JPA, and Kafka IT -- green twice, with negatives covered.



JWT/RBAC Stubs

AGENT/MANAGER roles annotated now; Lab 51 hardens deny-by-default.

MODULE FLOW RECAP

1

1. Select Slice

2

2. Build Layers

3

3. Publish Event

4

4. Consume Safely

5

5. Test & Document

KEY TAKEAWAY A vertical slice is graded end to end: API evidence, persistence proof, a versioned event, tests, and a documented failure path -- all six, not just the happy path.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of layered architecture, contracts, status codes, and idempotent consumers.

1. What must never appear as a controller method's return type?

- A. A DTO record
- B. A JPA entity**
- C. A UUID
- D. A ResponseEntity

3. Why must a Kafka consumer for this event be idempotent?

- A. Kafka guarantees exactly-once delivery
- B. At-least-once delivery makes duplicates normal**
- C. Idempotency is only needed for GET requests
- D. Consumers never receive duplicates

2. What status code should a successful interaction create return?

- A. 200
- B. 201**
- C. 204
- D. 302

4. What must NEVER happen when a request fails validation?

- A. Return 400 Problem Details
- B. Persist the row and publish the event anyway**
- C. Reject the request before it reaches the service
- D. Log the correlation id

KEY TAKEAWAY Entities stay internal; 201 on create; consumers must be idempotent; validation failures never persist or publish.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on testing discipline, fixtures, partitioning, and the merge gate.

5. What must a service unit test assert, per this module's testing rule?

- A. Only that no exception was thrown
- B. A concrete domain outcome, like the published event's payload**
- C. Nothing -- coverage percentage is enough
- D. The HTTP status code only

7. What must the Kafka partition key be for customer interaction events?

- A. A random UUID
- B. The event timestamp
- C. The customer id**
- D. The correlation id

6. Which fixture ID triggers the not-found path in Lab 49 tests?

- A. CUS-1001
- B. CUS-1002
- C. CUS-9999**
- D. lab-request-001

8. What is required before a merge of the interaction slice, per this module's rule?

- A. A single successful curl command
- B. API evidence, persistence proof, versioned event proof, tests, and a documented failure path**
- C. Only a green mvn test run
- D. Instructor sign-off with no evidence

KEY TAKEAWAY Assert real outcomes, not just 'no exception'; CUS-9999 is the not-found fixture; partition by customer id; merge requires all five evidence types.

"A vertical slice beats five disconnected demos -- API, database, and event, all working together."

MODULE 49 COMPLETE

Capstone Backend and Messaging

You can now implement a validated REST API, a transactional service, a versioned Kafka event, a resilient consumer, and the tests that prove all of it works -- CAP-12, built end to end against the Module 48 architecture.